

Reporte Ejecutivo

Proyecto Final - Machine Learning con PySpark y Docker - 2026-I

Estudiante: Sergio Huertas

Repositorio: github.com/chechitoooo/proyecto_final_Huertas_Sergio

1. Resumen ejecutivo

Este proyecto trabaja con dos fuentes de datos colombianas. La primera es el dataset TUSO del catastro de Bogota, que tiene 1.060.001 predios con tres columnas: codigo del lote, tipo de uso y area en metros cuadrados. La segunda es un corpus de 204 comentarios ciudadanos sobre alcaldias, ministerios, TransMilenio y servicios publicos. Los hallazgos principales son tres: el 68,1% de los predios de Bogota son de uso residencial con un area promedio de 187 metros cuadrados; el area del predio por si sola explica mas del 95% de la prediccion en los modelos de clasificacion, con un AUC de 0,79; y en el analisis de sentimiento, el modelo pre-entrenado de HuggingFace entiende sarcasmo y ambigüedad que el enfoque clasico de TF-IDF no logra capturar.

2. Que hice en cada bloque y por que

Bloque 1. Analisis exploratorio sobre TUSO. Cargue el dataset en PySpark (1.060.001 filas) y lo explore con seis tareas. Clasifique los predios por tamaño (pequeno, mediano, grande), calcule estadísticas descriptivas, revise si habia nulos o duplicados (no habia) e identifique valores atipicos. Tambien hice dos graficos con Matplotlib: un histograma de areas y un grafico de barras por tipo de uso. Use PySpark en vez de pandas porque un millon de registros no caben comoditos en memoria local.

Bloque 2. Machine learning sobre TUSO. Construi un pipeline con StringIndexer, OneHotEncoder, VectorAssembler y StandardScaler. Eleji StandardScaler en lugar de MinMaxScaler porque los predios tienen areas enormes (hasta 2,8 millones de m2) que aplastarian la escala del MinMax, y porque PCA necesita datos centrados en cero. En la parte no supervisada aplique PCA y K-Means con el metodo del codo (9 clusters). En la parte supervisada defini una variable objetivo binaria: predio residencial (si USOTUSO es 1) o no residencial (el resto). Entrene Regresion Logistica y Random Forest, reporte metricas (accuracy, precision, recall, F1 y AUC), hice matriz de confusion y extraje la importancia de variables del Random Forest. Por ultimo corri validacion cruzada con CrossValidator sobre 9 combinaciones de hiperparametros.

Bloque 3. NLP sobre corpus institucional. Arme un corpus de 204 documentos simulando tweets reales sobre instituciones publicas colombianas (TransMilenio, alcaldias, ministerios, policia, salud). Tokenice con RegexpTokenizer y filtre 179 stop words en espanol. Reporte estadísticas del corpus: total de tokens, vocabulario, TTR y cuantas palabras aparecen una sola vez (hapax legomena). Vectorice con CountVectorizer mas IDF y compare los rankings de frecuencia cruda contra TF-IDF. Luego use el modelo pre-entrenado de HuggingFace (robertuito, de la Universidad de Buenos Aires) para etiquetar el corpus como positivo o negativo. Sobre esas etiquetas entrene Regresion Logistica con vectores TF-IDF. Por ultimo compare los dos enfoques con 8 casos dificiles de sarcasmo, frases ambiguas y negaciones.

3. Resultados clave

Hallazgo 1 (Bloque 1). De los 1.060.001 predios, 721.963 (el 68,1%) son de uso residencial, con un area promedio de 187,13 m2. La media general del dataset es 295,56 m2 pero la mediana es apenas 158,7 m2, lo que muestra que unos pocos predios enormes jalan la media hacia arriba. El maximo registrado es 2.799.947,8 m2 y hay 6.046 predios con mas de 5.000 m2. Estos datos justifican usar logaritmo del area en el Bloque 2 para que los outliers no distorsionen los modelos.

Hallazgo 2 (Bloque 2). El area manda en la prediccion. En Random Forest, USOAREA aporta el 39,6% de importancia y log_area el 40,1%. Juntas suman el 79,7%. El sector geografico apenas aporta un 4,3%. Ambos modelos (Regresion Logistica y Random Forest) dan resultados parecidos: accuracy 0,78, F1 0,85 y AUC 0,78. Donde mas se equivoca el modelo es en los predios no residenciales: el 60,1% de los que no son residenciales los clasifica mal como si lo fueran. La validacion cruzada dio como ganador Random Forest con 100 arboles y profundidad maxima de 15 (AUC en CV de 0,7878).

Hallazgo 3 (Bloque 3). El modelo TF-IDF con Regresion Logistica saco accuracy de 0,60 y F1 de 0,71, pero esta muy sesgado hacia la clase negativa: acerto los 15 negativos del test pero erro los 10 positivos. Las palabras que mas pesan para positivo son "gracias", "historico" y "excelente". Las que mas pesan para negativo son "caos", "desastre", "corrupcion" y "prometio". En los 8 casos dificiles, HuggingFace acerto en frases con sarcasmo como "Que maravilla, otro dia sin agua" (la detecto como negativa) mientras que TF-IDF no pudo con ese tipo de lenguaje. El corpus tiene un 71% de hapax legomena, o sea que casi tres de cada cuatro palabras distintas aparecen una sola vez, algo muy tipico de tweets cortos.

4. Como se conectan los tres bloques

Los tres bloques cuentan una historia. El Bloque 1 me obligo a conocer los datos antes de modelar: vi que el area tiene una cola muy larga hacia la derecha y que el 68% de los predios son residenciales. Esa observacion me llevo a aplicar logaritmo en el Bloque 2, y funciono: con solo cuatro variables el Random Forest llego a un AUC de 0,79. Ademas el Bloque 2 me mostro que el area lo explica casi todo (80% de importancia) y que el sector geografico no aporta casi nada (4%). Eso es valioso para alguien que trabaje en catastro: no necesita datos complejos para clasificar predios, con el area basta. El Bloque 3 me llevo al otro extremo: los textos. Ahí el area no sirve de nada y lo que importa es el contexto. Un modelo clasico como TF-IDF falla con sarcasmo y frases ambiguas; un transformer como robertuito las entiende. La conclusion practica es que no hay un unico modelo que sirva para todo. Si voy a hacer avaluos catastrales, necesito un modelo interpretable como Random Forest, donde pueda explicar por que clasifique cada predio. Si voy a monitorear redes sociales para una alcaldia, necesito un transformer que entienda el tono real de los ciudadanos, aunque pese 500 MB y no pueda explicar cada decision. Un sistema maduro usaria los dos.

5. Limitaciones honestas de mi analisis

Muestras pequenas. En el Bloque 2 trabaje con 100.000 registros en vez del millon completo porque mi computador tiene 8 GB de RAM y apenas 1,6 GB libres. Los 9 clusters y las metricas reflejan esa muestra, no necesariamente todo Bogota. En el Bloque 3 el corpus es sintetico: lo arme yo simulando tweets reales, pero no fue validado por otras personas ni refleja toda la diversidad de opiniones que hay en redes sociales colombianas.

Etiquetas prestadas. Las etiquetas de sentimiento del Bloque 3 las puso robertuito, no un humano. Si el modelo pre-entrenado tiene sesgos (por ejemplo, clasificar como neutro algo que un colombiano leeria como negativo), ese sesgo se transmite al clasificador TF-IDF. Para estar seguros habria que anotar manualmente al menos 500 ejemplos y comparar.

Pocas variables en TUSO. El dataset solo trae tres columnas. Si tuviera datos como el avaluo catastral, el estrato, el año de construcción o coordenadas geográficas (todas disponibles en el IDECA), los modelos del Bloque 2 probablemente superarían un AUC de 0,85 sin mucho esfuerzo.

6. Que le diría a alguien que quiera seguir este trabajo

Use el Dockerfile que deje en el repositorio. Con docker-compose up se levanta Jupyter con Spark 3.5.0 y todas las dependencias listas. Ejecute los notebooks en orden: primero el de EDA, luego el de ML y por último el de NLP.

Para mejorar los resultados: reemplace el corpus sintético del Bloque 3 por tweets reales, mínimo 2.000. Cruce TUSO con otras fuentes del IDECA para meter más variables al modelo. Pruebe XGBoost de Spark ML, que en datos tabulares suele ganarle a Random Forest. Y si tiene acceso a un cluster con varios núcleos, corra PCA y K-Means sobre el dataset completo, no sobre la muestra. Para producción, una buena estrategia es usar HuggingFace para etiquetar datos nuevos automáticamente y luego reentrenar un modelo TF-IDF ligero que corra rápido y barato.